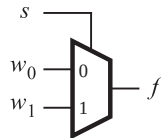


6.1 MULTIPLEXERS

Multiplexers were introduced briefly in Chapters 2 and 3. A multiplexer circuit has a number of data inputs, one or more select inputs, and one output. It passes the signal value on one of the data inputs to the output. The data input is selected by the values of the select inputs. Figure 6.1 shows a 2-to-1 multiplexer. Part (a) gives the symbol commonly used. The *select* input, s , chooses as the output of the multiplexer either input w_0 or w_1 . The multiplexer's functionality can be described in the form of a truth table as shown in part (b) of the figure. Part (c) gives a sum-of-products implementation of the 2-to-1 multiplexer, and part (d) illustrates how it can be constructed with transmission gates.

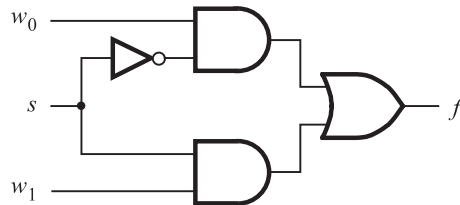
Figure 6.2a depicts a larger multiplexer with four data inputs, $w_0, \dots, w_3$, and two select inputs, s_1 and s_0 . As shown in the truth table in part (b) of the figure, the two-bit number represented by s_1s_0 selects one of the data inputs as the output of the multiplexer.



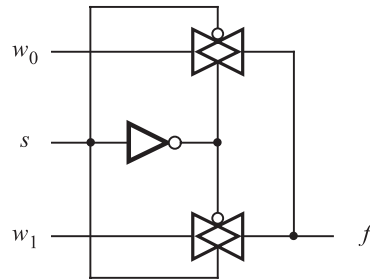
(a) Graphical symbol

s	f
0	w_0
1	w_1

(b) Truth table



(c) Sum-of-products circuit



(d) Circuit with transmission gates

Figure 6.1 A 2-to-1 multiplexer.

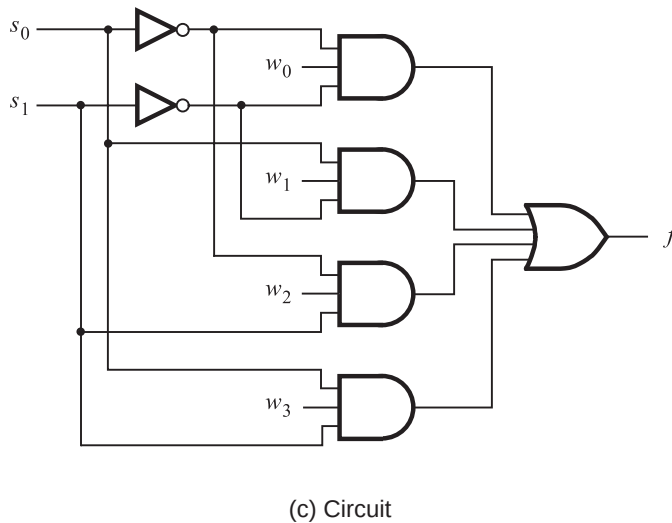
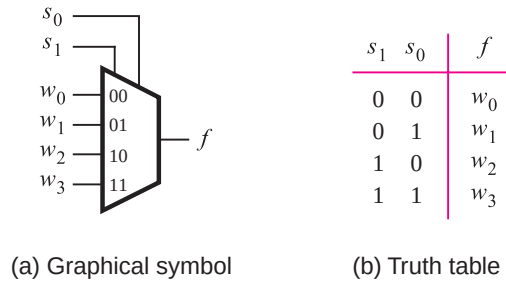


Figure 6.2 A 4-to-1 multiplexer.

A sum-of-products implementation of the 4-to-1 multiplexer appears in Figure 6.2c. It realizes the multiplexer function

$$f = \bar{s}_1 \bar{s}_0 w_0 + \bar{s}_1 s_0 w_1 + s_1 \bar{s}_0 w_2 + s_1 s_0 w_3$$

It is possible to build larger multiplexers using the same approach. Usually, the number of data inputs, n , is an integer power of two. A multiplexer that has n data inputs, $w_0, \dots, w_{n-1}$, requires $\lceil \log_2 n \rceil$ select inputs. Larger multiplexers can also be constructed from smaller multiplexers. For example, the 4-to-1 multiplexer can be built using three 2-to-1 multiplexers as illustrated in Figure 6.3. If the 4-to-1 multiplexer is implemented using transmission gates, then the structure in this figure is always used. Figure 6.4 shows how a 16-to-1 multiplexer is constructed with five 4-to-1 multiplexers.

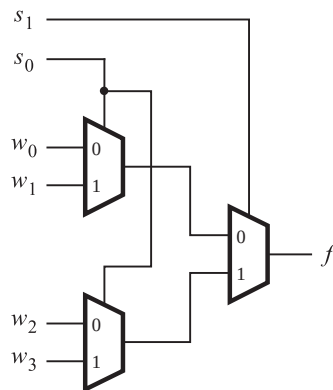


Figure 6.3 Using 2-to-1 multiplexers to build a 4-to-1 multiplexer.

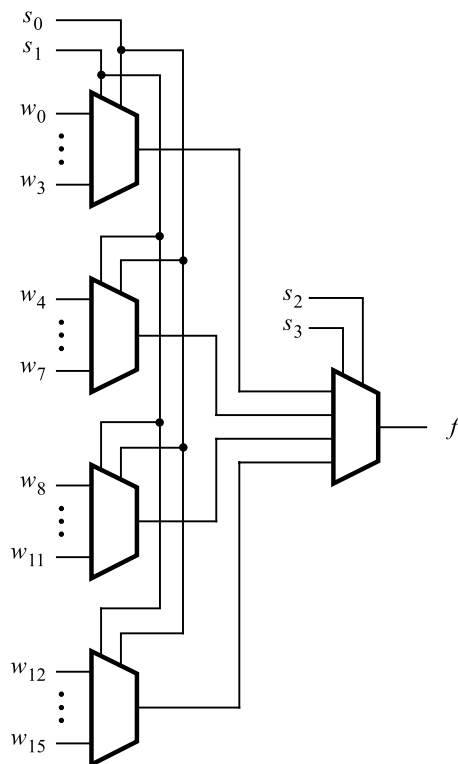
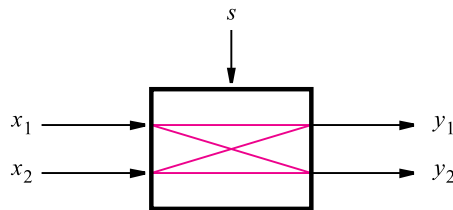


Figure 6.4 A 16-to-1 multiplexer.

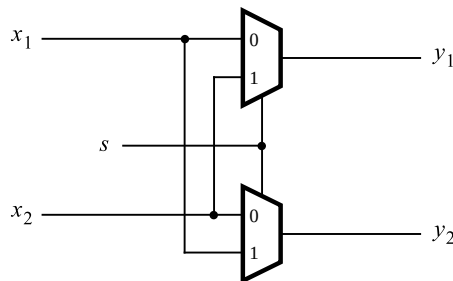
Figure 6.5 shows a circuit that has two inputs, x_1 and x_2 , and two outputs, y_1 and y_2 . As indicated by the blue lines, the function of the circuit is to allow either of its inputs to be connected to either of its outputs, under the control of another input, s . A circuit that has n inputs and k outputs, whose sole function is to provide a capability to connect any input to any output, is usually referred to as an $n \times k$ crossbar switch. Crossbars of various sizes can be created, with different numbers of inputs and outputs. When there are two inputs and two outputs, it is called a 2×2 crossbar.

Example 6.1

Figure 6.5b shows how the 2×2 crossbar can be implemented using 2-to-1 multiplexers. The multiplexer select inputs are controlled by the signal s . If $s = 0$, the crossbar connects x_1 to y_1 and x_2 to y_2 , while if $s = 1$, the crossbar connects x_1 to y_2 and x_2 to y_1 . Crossbar switches are useful in many practical applications in which it is necessary to be able to connect one set of wires to another set of wires, where the connection pattern changes from time to time.



(a) A 2×2 crossbar switch



(b) Implementation using multiplexers

Figure 6.5 A practical application of multiplexers.

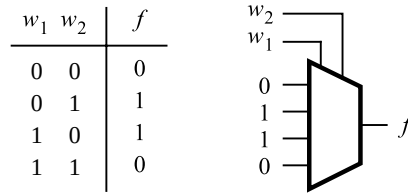
6.1.1 SYNTHESIS OF LOGIC FUNCTIONS USING MULTIPLEXERS

Multiplexers are useful in many practical applications, such as those described above. They can also be used in a more general way to synthesize logic functions. Consider the example in Figure 6.7a. The truth table defines the function $f = w_1 \oplus w_2$. This function can be implemented by a 4-to-1 multiplexer in which the values of f in each row of the truth table are connected as constants to the multiplexer data inputs. The multiplexer select inputs are driven by w_1 and w_2 . Thus for each valuation of w_1w_2 , the output f is equal to the function value in the corresponding row of the truth table.

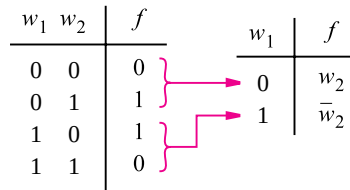
The above implementation is straightforward, but it is not very efficient. A better implementation can be derived by manipulating the truth table as indicated in Figure 6.7b, which allows f to be implemented by a single 2-to-1 multiplexer. One of the input signals, w_1 in this example, is chosen as the select input of the 2-to-1 multiplexer. The truth table is redrawn to indicate the value of f for each value of w_1 . When $w_1 = 0$, f has the same value as input w_2 , and when $w_1 = 1$, f has the value of $\bar{w}_2$. The circuit that implements this truth table is given in Figure 6.7c. This procedure can be applied to synthesize a circuit that implements any logic function.

Figure 6.8a gives the truth table for the three-input majority function, and it shows how the truth table can be modified to implement the function using a 4-to-1 multiplexer. Any two of the three inputs may be chosen as the multiplexer select inputs. We have chosen w_1 and w_2 for this purpose, resulting in the circuit in Figure 6.8b.

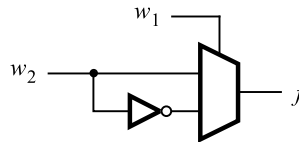
Example 6.3



(a) Implementation using a 4-to-1 multiplexer



(b) Modified truth table



(c) Circuit

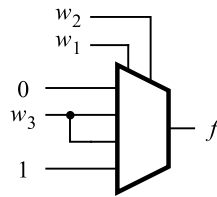
Figure 6.7 Synthesis of a logic function using multiplexers.

Example 6.4 Figure 6.9a indicates how the function $f = w_1 \oplus w_2 \oplus w_3$ can be implemented using 2-to-1 multiplexers. When $w_1 = 0$, f is equal to the XOR of w_2 and w_3 , and when $w_1 = 1$, f is the XNOR of w_2 and w_3 . The left multiplexer in the circuit produces $w_2 \oplus w_3$, using the result from Figure 6.7, and the right multiplexer uses the value of w_1 to select either $w_2 \oplus w_3$ or its complement. Note that we could have derived this circuit directly by writing the function as $f = (w_2 \oplus w_3) \oplus w_1$.

Figure 6.10 gives an implementation of the three-input XOR function using a 4-to-1 multiplexer. Choosing w_1 and w_2 for the select inputs results in the circuit shown.

w_1	w_2	w_3	f		w_1	w_2	f
0	0	0	0	}	0	0	0
0	0	1	0		0	1	w_3
0	1	0	0		1	0	w_3
0	1	1	1		1	1	1
1	0	0	0	}			
1	0	1	1				
1	1	0	1				
1	1	1	1				

(a) Modified truth table

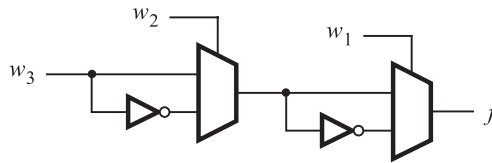


(b) Circuit

Figure 6.8 Implementation of the three-input majority function using a 4-to-1 multiplexer.

w_1	w_2	w_3	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(a) Truth table



(b) Circuit

Figure 6.9 Three-input XOR implemented with 2-to-1 multiplexers.

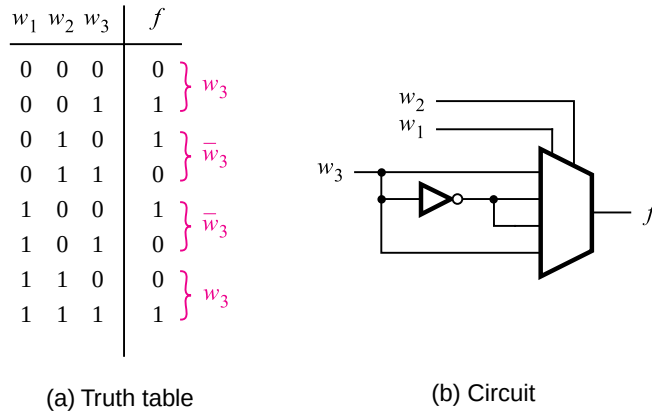


Figure 6.10 Three-input XOR implemented with a 4-to-1 multiplexer.

6.1.2 MULTIPLEXER SYNTHESIS USING SHANNON'S EXPANSION

Figures 6.8 through 6.10 illustrate how truth tables can be interpreted to implement logic functions using multiplexers. In each case the inputs to the multiplexers are the constants 0 and 1, or some variable or its complement. Besides using such simple inputs, it is possible to connect more complex circuits as inputs to a multiplexer, allowing functions to be synthesized using a combination of multiplexers and other logic gates. Suppose that we want to implement the three-input majority function in Figure 6.8 using a 2-to-1 multiplexer in this way. Figure 6.11 shows an intuitive way of realizing this function. The truth table can be modified as shown on the right. If $w_1 = 0$, then $f = w_2 w_3$, and if $w_1 = 1$, then $f = w_2 + w_3$. Using w_1 as the select input for a 2-to-1 multiplexer leads to the circuit in Figure 6.11b.

This implementation can be derived using algebraic manipulation as follows. The function in Figure 6.11a is expressed in sum-of-products form as

$$f = \bar{w}_1 w_2 w_3 + w_1 \bar{w}_2 w_3 + w_1 w_2 \bar{w}_3 + w_1 w_2 w_3$$

It can be manipulated into

$$\begin{aligned} f &= \bar{w}_1 (w_2 w_3) + w_1 (\bar{w}_2 w_3 + w_2 \bar{w}_3 + w_2 w_3) \\ &= \bar{w}_1 (w_2 w_3) + w_1 (w_2 + w_3) \end{aligned}$$

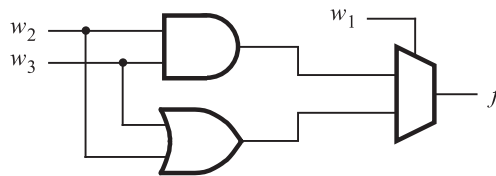
which corresponds to the circuit in Figure 6.11b.

Multiplexer implementations of logic functions require that a given function be decomposed in terms of the variables that are used as the select inputs. This can be accomplished by means of a theorem proposed by Claude Shannon [1].

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

w_1	f
0	$w_2 w_3$
1	$w_2 + w_3$

(a) Truth table



(b) Circuit

Figure 6.11 The three-input majority function implemented using a 2-to-1 multiplexer.

Shannon's Expansion Theorem

Any Boolean function $f(w_1, \dots, w_n)$ can be written in the form

$$f(w_1, w_2, \dots, w_n) = \bar{w}_1 \cdot f(0, w_2, \dots, w_n) + w_1 \cdot f(1, w_2, \dots, w_n)$$

This expansion can be done in terms of any of the n variables. We will leave the proof of the theorem as an exercise for the reader (see problem 6.9).

To illustrate its use, we can apply the theorem to the three-input majority function, which can be written as

$$f(w_1, w_2, w_3) = w_1 w_2 + w_1 w_3 + w_2 w_3$$

Expanding this function in terms of w_1 gives

$$f = \bar{w}_1(w_2 w_3) + w_1(w_2 + w_3)$$

which is the expression that we derived above.

For the three-input XOR function, we have

$$\begin{aligned} f &= w_1 \oplus w_2 \oplus w_3 \\ &= \bar{w}_1 \cdot (w_2 \oplus w_3) + w_1 \cdot (\overline{w_2 \oplus w_3}) \end{aligned}$$

which gives the circuit in Figure 6.9b.

In Shannon's expansion the term $f(0, w_2, \dots, w_n)$ is called the *cofactor* of f with respect to $\bar{w}_1$; it is denoted in shorthand notation as $f_{\bar{w}_1}$. Similarly, the term $f(1, w_2, \dots, w_n)$ is called the cofactor of f with respect to w_1 , written f_{w_1} . Hence we can write

$$f = \bar{w}_1 f_{\bar{w}_1} + w_1 f_{w_1}$$

In general, if the expansion is done with respect to variable w_i , then f_{w_i} denotes $f(w_1, \dots, w_{i-1}, 1, w_{i+1}, \dots, w_n)$ and

$$f(w_1, \dots, w_n) = \bar{w}_i f_{\bar{w}_i} + w_i f_{w_i}$$

The complexity of the logic expression may vary, depending on which variable, w_i , is used, as illustrated in Example 6.5.

Example 6.5 For the function $f = \bar{w}_1 w_3 + w_2 \bar{w}_3$, decomposition using w_1 gives

$$\begin{aligned} f &= \bar{w}_1 f_{\bar{w}_1} + w_1 f_{w_1} \\ &= \bar{w}_1 (w_3 + w_2) + w_1 (w_2 \bar{w}_3) \end{aligned}$$

Using w_2 instead of w_1 produces

$$\begin{aligned} f &= \bar{w}_2 f_{\bar{w}_2} + w_2 f_{w_2} \\ &= \bar{w}_2 (\bar{w}_1 w_3) + w_2 (\bar{w}_1 + \bar{w}_3) \end{aligned}$$

Finally, using w_3 gives

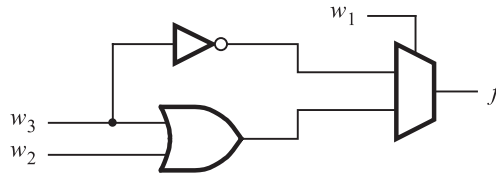
$$\begin{aligned} f &= \bar{w}_3 f_{\bar{w}_3} + w_3 f_{w_3} \\ &= \bar{w}_3 (w_2) + w_3 (\bar{w}_1) \end{aligned}$$

The results generated using w_1 and w_2 have the same cost, but the expression produced using w_3 has a lower cost. In practice, the CAD tools that perform decompositions of this type try a number of alternatives and choose the one that produces the best result.

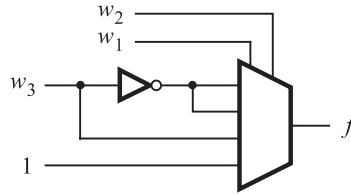
Shannon's expansion can be done in terms of more than one variable. For example, expanding a function in terms of w_1 and w_2 gives

$$\begin{aligned} f(w_1, \dots, w_n) &= \bar{w}_1 \bar{w}_2 \cdot f(0, 0, w_3, \dots, w_n) + \bar{w}_1 w_2 \cdot f(0, 1, w_3, \dots, w_n) \\ &\quad + w_1 \bar{w}_2 \cdot f(1, 0, w_3, \dots, w_n) + w_1 w_2 \cdot f(1, 1, w_3, \dots, w_n) \end{aligned}$$

This expansion gives a form that can be implemented using a 4-to-1 multiplexer. If Shannon's expansion is done in terms of all n variables, then the result is the canonical sum-of-products form, which was defined in section 2.6.1.



(a) Using a 2-to-1 multiplexer



(b) Using a 4-to-1 multiplexer

Figure 6.12 The circuits synthesized in Example 6.6.

Assume that we wish to implement the function

Example 6.6

$$f = \bar{w}_1 \bar{w}_3 + w_1 w_2 + w_1 w_3$$

using a 2-to-1 multiplexer and any other necessary gates. Shannon's expansion using w_1 gives

$$\begin{aligned} f &= \bar{w}_1 f_{\bar{w}_1} + w_1 f_{w_1} \\ &= \bar{w}_1 (\bar{w}_3) + w_1 (w_2 + w_3) \end{aligned}$$

The corresponding circuit is shown in Figure 6.12a. Assume now that we wish to use a 4-to-1 multiplexer instead. Further decomposition using w_2 gives

$$\begin{aligned} f &= \bar{w}_1 \bar{w}_2 f_{\bar{w}_1 \bar{w}_2} + \bar{w}_1 w_2 f_{\bar{w}_1 w_2} + w_1 \bar{w}_2 f_{w_1 \bar{w}_2} + w_1 w_2 f_{w_1 w_2} \\ &= \bar{w}_1 \bar{w}_2 (\bar{w}_3) + \bar{w}_1 w_2 (\bar{w}_3) + w_1 \bar{w}_2 (w_3) + w_1 w_2 (1) \end{aligned}$$

The circuit is shown in Figure 6.12b.

Consider the three-input majority function

Example 6.7

$$f = w_1 w_2 + w_1 w_3 + w_2 w_3$$

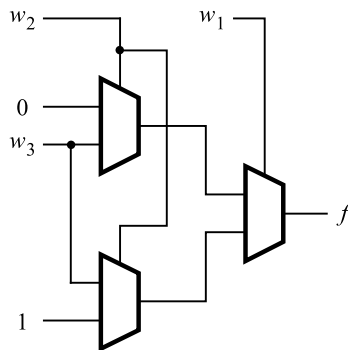


Figure 6.13 The circuit synthesized in Example 6.7.

We wish to implement this function using only 2-to-1 multiplexers. Shannon's expansion using w_1 yields

$$\begin{aligned} f &= \bar{w}_1(w_2w_3) + w_1(w_2 + w_3 + w_2w_3) \\ &= \bar{w}_1(w_2w_3) + w_1(w_2 + w_3) \end{aligned}$$

Let $g = w_2w_3$ and $h = w_2 + w_3$. Expansion of both g and h using w_2 gives

$$\begin{aligned} g &= \bar{w}_2(0) + w_2(w_3) \\ h &= \bar{w}_2(w_3) + w_2(1) \end{aligned}$$

The corresponding circuit is shown in Figure 6.13. It is equivalent to the 4-to-1 multiplexer circuit derived using a truth table in Figure 6.8.

6.2 DECODERS

Decoder circuits are used to decode encoded information. A binary decoder, depicted in Figure 6.15, is a logic circuit with n inputs and 2^n outputs. Only one output is asserted at a time, and each output corresponds to one valuation of the inputs. The decoder also has an enable input, En , that is used to disable the outputs; if $En = 0$, then none of the decoder outputs is asserted. If $En = 1$, the valuation of $w_{n-1} \cdots w_1 w_0$ determines which of the outputs is asserted. An n -bit binary code in which exactly one of the bits is set to 1 at a

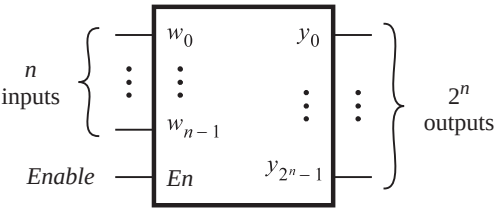


Figure 6.15 An n -to- 2^n binary decoder.

time is referred to as *one-hot encoded*, meaning that the single bit that is set to 1 is deemed to be “hot.” The outputs of a binary decoder are one-hot encoded.

A 2-to-4 decoder is given in Figure 6.16. The two data inputs are w_1 and w_0 . They represent a two-bit number that causes the decoder to assert one of the outputs $y_0, \dots, y_3$. Although a decoder can be designed to have either active-high or active-low outputs, in

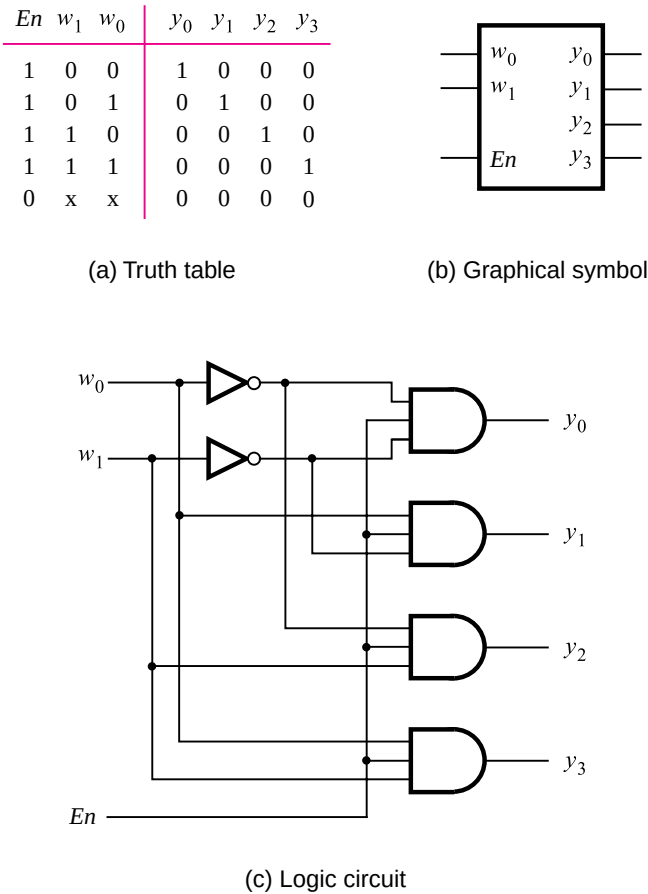


Figure 6.16 A 2-to-4 decoder.

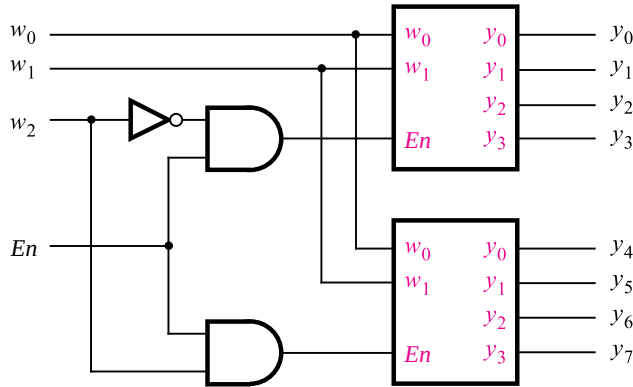


Figure 6.17 A 3-to-8 decoder using two 2-to-4 decoders.

Figure 6.16 active-high outputs are assumed. Setting the inputs w_1w_0 to 00, 01, 10, or 11 causes the output y_0 , y_1 , y_2 , or y_3 to be set to 1, respectively. A graphical symbol for the decoder is given in part (b) of the figure, and a logic circuit is shown in part (c).

Larger decoders can be built using the sum-of-products structure in Figure 6.16c, or else they can be constructed from smaller decoders. Figure 6.17 shows how a 3-to-8 decoder is built with two 2-to-4 decoders. The w_2 input drives the enable inputs of the two decoders. The top decoder is enabled if $w_2 = 0$, and the bottom decoder is enabled if $w_2 = 1$. This concept can be applied for decoders of any size. Figure 6.18 shows how five 2-to-4 decoders can be used to construct a 4-to-16 decoder. Because of its treelike structure, this type of circuit is often referred to as a *decoder tree*.

Decoders are useful for many practical purposes. In Figure 6.2c we showed the sum-of-products implementation of the 4-to-1 multiplexer, which requires AND gates to distinguish the four different valuations of the select inputs s_1 and s_0 . Since a decoder evaluates the values on its inputs, it can be used to build a multiplexer as illustrated in Figure 6.19. The enable input of the decoder is not needed in this case, and it is set to 1. The four outputs of the decoder represent the four valuations of the select inputs.

Example 6.9

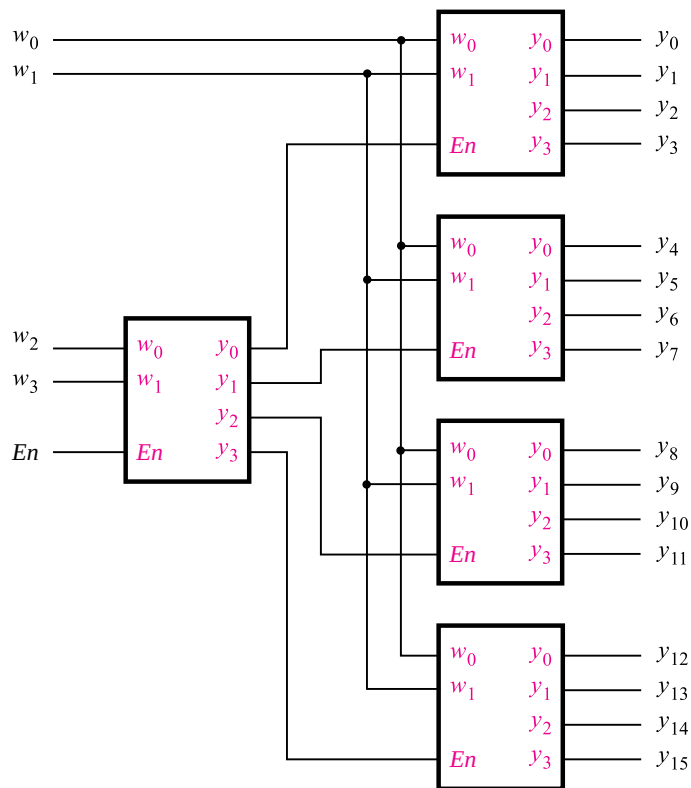


Figure 6.18 A 4-to-16 decoder built using a decoder tree.

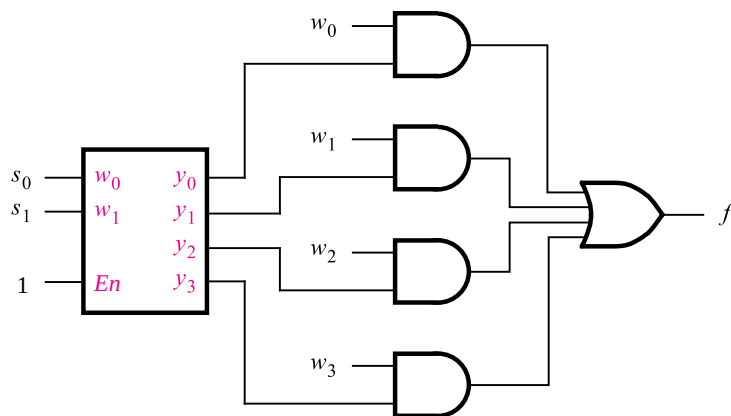


Figure 6.19 A 4-to-1 multiplexer built using a decoder.

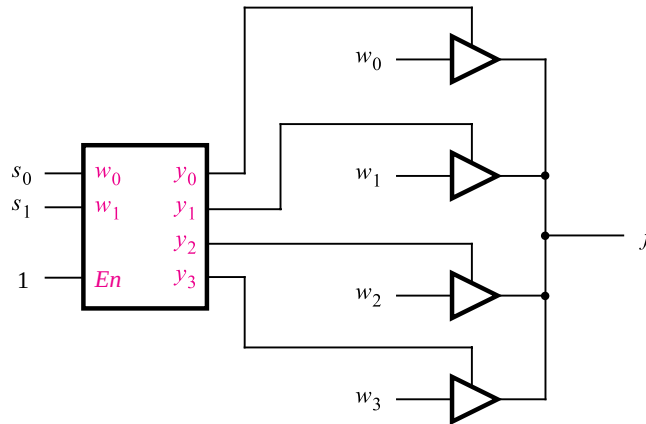


Figure 6.20 A 4-to-1 multiplexer built using a decoder and tri-state buffers.

buffers. Hence multiplexers must be implemented in the sum-of-products form using the lookup tables (see Example 6.30).

6.2.1 DEMULTIPLEXERS

We showed in section 6.1 that a multiplexer has one output, n data inputs, and $\lceil \log_2 n \rceil$ select inputs. The purpose of the multiplexer circuit is to *multiplex* the n data inputs onto the single data output under control of the select inputs. A circuit that performs the opposite function, namely, placing the value of a single data input onto multiple data outputs, is called a *demultiplexer*. The demultiplexer can be implemented using a decoder circuit. For example, the 2-to-4 decoder in Figure 6.16 can be used as a 1-to-4 demultiplexer. In this case the En input serves as the data input for the demultiplexer, and the y_0 to y_3 outputs are the data outputs. The valuation of w_1w_0 determines which of the outputs is set to the value of En . To see how the circuit works, consider the truth table in Figure 6.16a. When $En = 0$, all the outputs are set to 0, including the one selected by the valuation of w_1w_0 . When $En = 1$, the valuation of w_1w_0 sets the appropriate output to 1.

In general, an n -to- 2^n decoder circuit can be used as a 1-to- n demultiplexer. However, in practice decoder circuits are used much more often as decoders rather than as demultiplexers. In many applications the decoder's En input is not actually needed; hence it can be omitted. In this case the decoder always asserts one of its data outputs, $y_0, \dots, y_{2^n-1}$, according to the valuation of the data inputs, $w_{n-1} \cdots w_0$. Example 6.11 uses a decoder that does not have the En input.

Example 6.11 One of the most important applications of decoders is in memory blocks, which are used to store information. Such memory blocks are included in digital systems, such as computers, where there is a need to store large amounts of information electronically. One type of memory block is called a *read-only memory* (ROM). A ROM consists of a collection of storage cells, where each cell permanently stores a single logic value, either 0 or 1. Figure 6.21 shows an example of a ROM block. The storage cells are arranged in 2^m rows with n cells per row. Thus each row stores n bits of information. The location of each row in the ROM is identified by its *address*. In the figure the row at the top of the ROM has address 0, and the row at the bottom has address $2^m - 1$. The information stored in the rows can be accessed by asserting the select lines, Sel_0 to Sel_{2^m-1} . As shown in the figure, a decoder with m inputs and 2^m outputs is used to generate the signals on the select lines. Since the inputs to the decoder choose the particular address (row) selected, they are called the *address* lines. The information stored in the row appears on the data outputs of the ROM, $d_{n-1}, \dots, d_0$, which are called the *data* lines. Figure 6.21 shows that each data line has an associated tri-state buffer that is enabled by the ROM input named *Read*. To access, or *read*, data from the ROM, the address of the desired row is placed on the address lines and *Read* is set to 1.

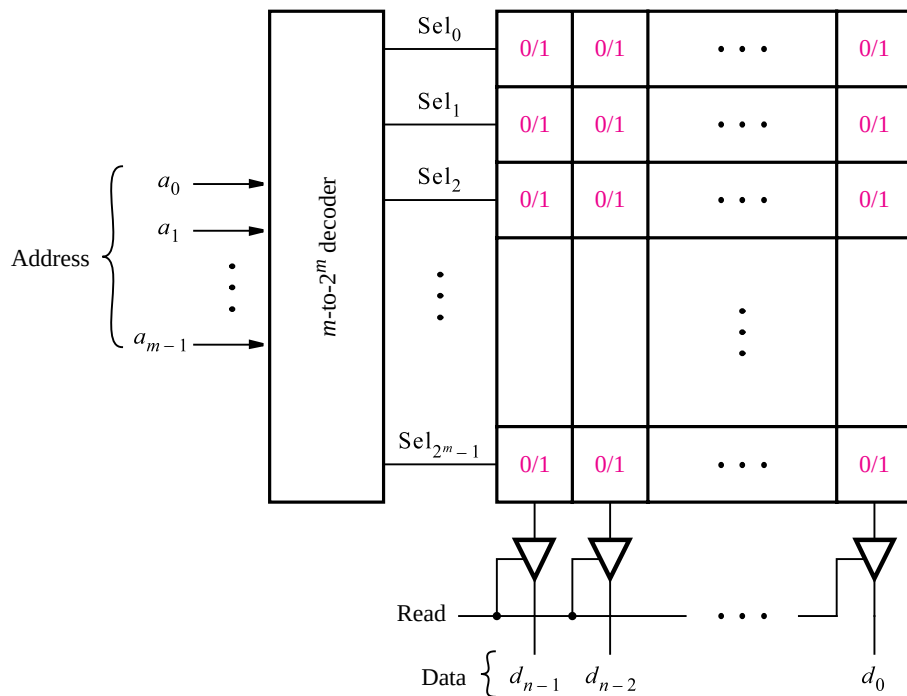


Figure 6.21 A $2^m \times n$ read-only memory (ROM) block.

Many different types of memory blocks exist. In a ROM the stored information can be read out of the storage cells, but it cannot be changed (see problem 6.32). Another type of ROM allows information to be both read out of the storage cells and stored, or *written*, into them. Reading its contents is the normal operation, whereas writing requires a special procedure. Such a memory block is called a programmable ROM (PROM). The storage cells in a PROM are usually implemented using EEPROM transistors. We discussed EEPROM transistors in section 3.10 to show how they are used in PLDs. Other types of memory blocks are discussed in section 10.1.

6.3 ENCODERS

An encoder performs the opposite function of a decoder. It encodes given information into a more compact form.

6.3.1 BINARY ENCODERS

A *binary encoder* encodes information from 2^n inputs into an n -bit code, as indicated in Figure 6.22. Exactly one of the input signals should have a value of 1, and the outputs present the binary number that identifies which input is equal to 1. The truth table for a 4-to-2 encoder is provided in Figure 6.23a. Observe that the output y_0 is 1 when either input w_1 or w_3 is 1, and output y_1 is 1 when input w_2 or w_3 is 1. Hence these outputs can be generated by the circuit in Figure 6.23b. Note that we assume that the inputs are one-hot encoded. All input patterns that have multiple inputs set to 1 are not shown in the truth table, and they are treated as don't-care conditions.

Encoders are used to reduce the number of bits needed to represent given information. A practical use of encoders is for transmitting information in a digital system. Encoding the information allows the transmission link to be built using fewer wires. Encoding is also useful if information is to be stored for later use because fewer bits need to be stored.

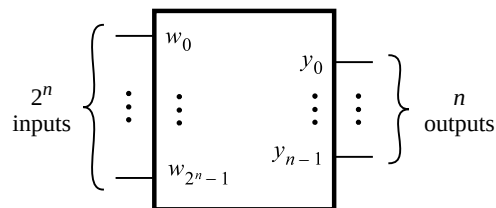
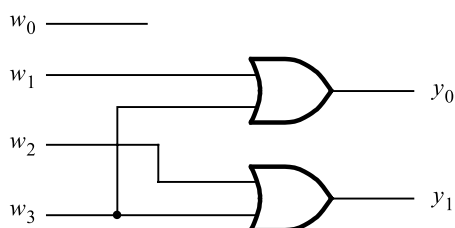


Figure 6.22 A 2^n -to- n binary encoder.

w_3	w_2	w_1	w_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

(a) Truth table



(b) Circuit

Figure 6.23 A 4-to-2 binary encoder.

6.3.2 PRIORITY ENCODERS

Another useful class of encoders is based on the priority of input signals. In a *priority encoder* each input has a priority level associated with it. The encoder outputs indicate the active input that has the highest priority. When an input with a high priority is asserted, the other inputs with lower priority are ignored. The truth table for a 4-to-2 priority encoder is shown in Figure 6.24. It assumes that w_0 has the lowest priority and w_3 the highest. The outputs y_1 and y_0 represent the binary number that identifies the highest priority input set to 1. Since it is possible that none of the inputs is equal to 1, an output, z , is provided to indicate this condition. It is set to 1 when at least one of the inputs is equal to 1. It is set to

w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

Figure 6.24 Truth table for a 4-to-2 priority encoder.

0 when all inputs are equal to 0. The outputs y_1 and y_0 are not meaningful in this case, and hence the first row of the truth table can be treated as a don't-care condition for y_1 and y_0 .

The behavior of the priority encoder is most easily understood by first considering the last row in the truth table. It specifies that if input w_3 is 1, then the outputs are set to $y_1y_0 = 11$. Because w_3 has the highest priority level, the values of inputs w_2 , w_1 , and w_0 do not matter. To reflect the fact that their values are irrelevant, w_2 , w_1 , and w_0 are denoted by the symbol x in the truth table. The second-last row in the truth table stipulates that if $w_2 = 1$, then the outputs are set to $y_1y_0 = 10$, but only if $w_3 = 0$. Similarly, input w_1 causes the outputs to be set to $y_1y_0 = 01$ only if both w_3 and w_2 are 0. Input w_0 produces the outputs $y_1y_0 = 00$ only if w_0 is the only input that is asserted.

A logic circuit that implements the truth table can be synthesized by using the techniques developed in Chapter 4. However, a more convenient way to derive the circuit is to define a set of intermediate signals, $i_0, \dots, i_3$, based on the observations above. Each signal, i_k , is equal to 1 only if the input with the same index, w_k , represents the highest-priority input that is set to 1. The logic expressions for $i_0, \dots, i_3$ are

$$i_0 = \overline{w_3}\overline{w_2}\overline{w_1}w_0$$

$$i_1 = \overline{w_3}\overline{w_2}w_1$$

$$i_2 = \overline{w_3}w_2$$

$$i_3 = w_3$$

Using the intermediate signals, the rest of the circuit for the priority encoder has the same structure as the binary encoder in Figure 6.23, namely

$$y_0 = i_1 + i_3$$

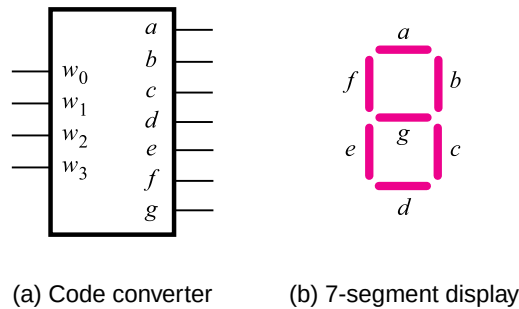
$$y_1 = i_2 + i_3$$

The output z is given by

$$z = i_0 + i_1 + i_2 + i_3$$

6.4 CODE CONVERTERS

The purpose of the decoder and encoder circuits is to convert from one type of input encoding to a different output encoding. For example, a 3-to-8 binary decoder converts from a binary number on the input to a one-hot encoding at the output. An 8-to-3 binary encoder performs the opposite conversion. There are many other possible types of code converters. One common example is a BCD-to-7-segment decoder, which converts one binary-coded decimal (BCD) digit into information suitable for driving a digit-oriented display. As illustrated in Figure 6.25a, the circuit converts the BCD digit into seven signals that are used to drive the segments in the display. Each segment is a small light-emitting diode (LED), which glows when driven by an electrical signal. The segments are labeled from a to g in the figure. The truth table for the BCD-to-7-segment decoder is given in Figure 6.25c. For each valuation of the inputs $w_3, \dots, w_0$, the seven outputs are set to



w_3	w_2	w_1	w_0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

(c) Truth table

Figure 6.25 A BCD-to-7-segment display code converter.

display the appropriate BCD digit. Note that the last 6 rows of a complete 16-row truth table are not shown. They represent don't-care conditions because they are not legal BCD codes and will never occur in a circuit that deals with BCD data. A circuit that implements the truth table can be derived using the synthesis techniques discussed in Chapter 4. Finally, we should note that although the word *decoder* is traditionally used for this circuit, a more appropriate term is *code converter*. The term *decoder* is more appropriate for circuits that produce one-hot encoded outputs.

6.5 ARITHMETIC COMPARISON CIRCUITS

Chapter 5 presented arithmetic circuits that perform addition, subtraction, and multiplication of binary numbers. Another useful type of arithmetic circuit compares the relative sizes of two binary numbers. Such a circuit is called a *comparator*. This section considers the

design of a comparator that has two n -bit inputs, A and B , which represent unsigned binary numbers. The comparator produces three outputs, called $AeqB$, $AgtB$, and $AltB$. The $AeqB$ output is set to 1 if A and B are equal. The $AgtB$ output is 1 if A is greater than B , and the $AltB$ output is 1 if A is less than B .

The desired comparator can be designed by creating a truth table that specifies the three outputs as functions of A and B . However, even for moderate values of n , the truth table is large. A better approach is to derive the comparator circuit by considering the bits of A and B in pairs. We can illustrate this by a small example, where $n = 4$.

Let $A = a_3a_2a_1a_0$ and $B = b_3b_2b_1b_0$. Define a set of intermediate signals called i_3, i_2, i_1 , and i_0 . Each signal, i_k , is 1 if the bits of A and B with the same index are equal. That is, $i_k = a_k \oplus b_k$. The comparator's $AeqB$ output is then given by

$$AeqB = i_3i_2i_1i_0$$

An expression for the $AgtB$ output can be derived by considering the bits of A and B in the order from the most-significant bit to the least-significant bit. The first bit-position, k , at which a_k and b_k differ determines whether A is less than or greater than B . If $a_k = 0$ and $b_k = 1$, then $A < B$. But if $a_k = 1$ and $b_k = 0$, then $A > B$. The $AgtB$ output is defined by

$$AgtB = a_3\bar{b}_3 + i_3a_2\bar{b}_2 + i_3i_2a_1\bar{b}_1 + i_3i_2i_1a_0\bar{b}_0$$

The i_k signals ensure that only the first digits, considered from the left to the right, of A and B that differ determine the value of $AgtB$.

The $AltB$ output can be derived by using the other two outputs as

$$AltB = \overline{AeqB + AgtB}$$

A logic circuit that implements the four-bit comparator circuit is shown in Figure 6.26. This approach can be used to design a comparator for any value of n .

Comparator circuits, like most logic circuits, can be designed in different ways. Another approach for designing a comparator circuit is presented in Example 5.10 in Chapter 5.

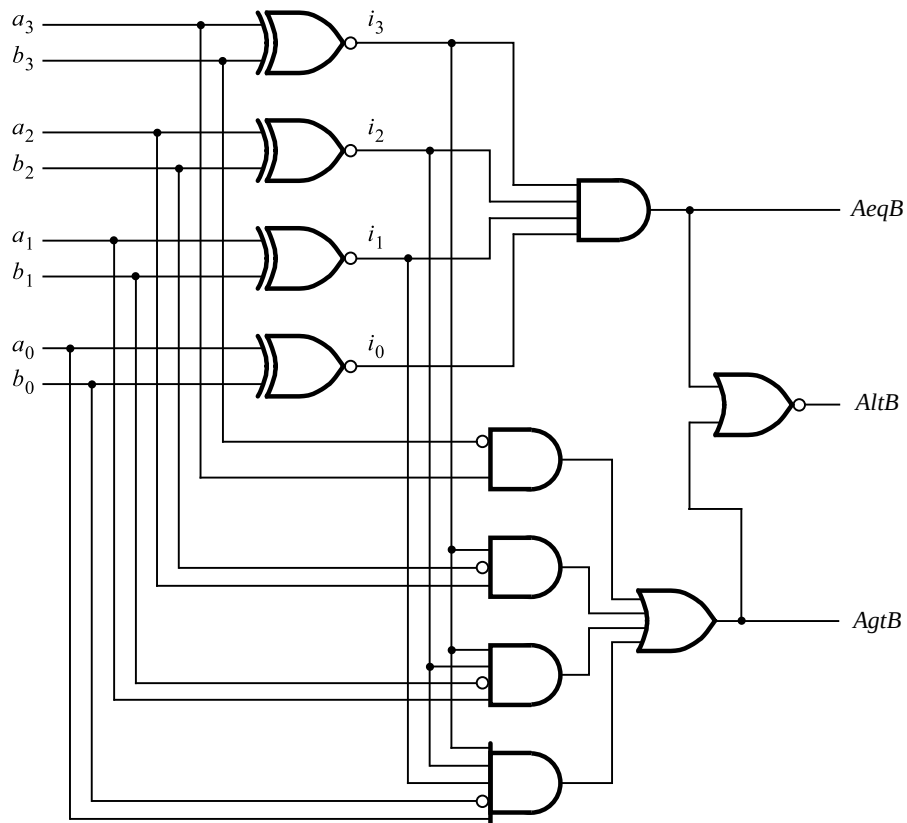


Figure 6.26 A four-bit comparator circuit.